

Name: Kayode Timileyin Nicholas

Matric Number: CYS/20/4924

DESIGN AND IMPLEMENTATION OF DEEP-BASED IDS USING LSTM

Supervisor: Dr. Akinyokun

CHAPTER 1: INTRODUCTION

1.1 Background of the Study

The proliferation of digital technologies and the increasing availability of internet connectivity have changed the nature of organizations in Nigeria, and indeed globally. Network infrastructure is now integral to key sectors such as banking, telecommunications, health and governance, providing a platform for seamless transfer of data and rapid deployment of services. Consequently, dependence on networked systems has, at the same time, exposed these organizations to increasing levels of cyber-attacks. Current cybercriminals employ a myriad of sophisticated methods to breach network perimeters, steal sensitive data and disrupt operations, with a frequency that shows an upward trend and a creativity that constantly pushes the frontiers. The use of network and system resources to carry out criminal activities, also known as cyber-crime or computer crime, is on the rise. Akinwale and Adeyemi (2020) have identified that incidents of cyber-crime have increased significantly in Nigeria, with a high proportion affecting government organizations and financial institutions. In financial sectors, financial loss due to computer fraud has accumulated to billions of Nigerian naira yearly.

In this regard, intrusion detection systems have become one of the most critical tools in an organization's cybersecurity arsenal; acting as the digital sentinels that monitor traffic on a network to identify suspicious activity and policy violations. Signature-based intrusion detection systems (IDS) identify malicious traffic by comparing incoming network packets against a database of previously identified attack signatures (Denning, 1987). Signature-based systems are only effective against attacks that are already well documented; the system's effectiveness drastically reduces when there is a new type of attack or when an existing attack has been

polymorphic. To effectively defend against this type of attack, signature-based IDS must constantly be updated, requiring human intervention which becomes resource-intensive and reactive. This deficiency has thus led researchers to investigate alternative intelligent intrusion detection systems that can learn from data, and adapt to future attack types without being re-programmed every time.

The Machine learning approach has brought the expected breakthrough. The algorithms, such as decision trees, support vector machines, random forests, and so forth, showed much better capability of detecting malicious activity by learning discriminating patterns in labelled training data (Othman et al., 2018). However, traditional classifiers still treated network features as independent observations and failed to take advantage of the temporal sequence structure of many attacks. Most attacks on a network system don't happen instantaneously, but rather across several timesteps and these types of attacks, for example the User-to-Root (U2R) and Remote-to-Local (R2L) attacks are usually executed in a sequence. The current study seeks to mitigate this problem and enhance performance of intrusion detection systems by using deep learning approaches that are capable of modelling temporal dependency.

Inspired by the human brain, deep learning has introduced neural networks whose architectures and behavior are modelled from biology. The Recurrent Neural Networks (RNN) model is well known for its capability of processing sequential data through the retention of memory states representing information from previous steps (Goodfellow et al., 2016). Long Short-Term Memory networks (LSTMs) were first proposed in Hochreiter & Schmidhuber (1997) and they present a special type of RNNs that overcomes the problem of vanishing gradient of typical RNNs through the implementation of special mechanisms namely; input gate, output gate and forget gate. This special gating mechanism enables LSTMs to model long range dependencies, and this ability has been exploited in various sequence classification tasks (Sherstinsky, 2020), with network intrusion detection being one of them.

This study will focus on the application of LSTMs to the design and implementation of a deep learning based intelligent IDS that can classify network traffic either as normal or malicious based on the learned temporal patterns on established benchmark datasets. This model will be developed using Python and powered by deep learning framework libraries such as TensorFlow/Keras and an alternative experiment can be carried out using Pytorch. The libraries such as Numpy, Pandas and Scikit-learn will be used for data pre-processing and feature engineering and results will be visualized using Matplotlib and Seaborn libraries. NSL-KDD, CICIDS2017 and UNSW-NB15 benchmark datasets will be utilized to train and evaluate the designed IDS. This study also aims to enhance the existing cybersecurity approaches to help protect the Nigerian system.

1.2 Problem Statement

Network security is a critical concern for organisations in Nigeria and elsewhere, and intrusion detection systems sit at the heart of most defensive strategies. Despite decades of research and commercial development, existing IDS implementations continue to exhibit limitations that are difficult to overlook. Signature-based systems — which still dominate the commercial market — are reactive by design. They can only identify what they already know, rendering them largely ineffective against zero-day exploits and attack vectors that have not yet been catalogued in any signature database (Tavallaee et al., 2009). Given how rapidly the threat landscape evolves, this dependency on prior knowledge is a structural vulnerability, not merely an inconvenience.

Machine learning has partially addressed these shortcomings, but introduces its own set of problems. Conventional classifiers such as support vector machines and ensemble methods typically treat network traffic observations as independent and identically distributed, thereby ignoring the temporal dependencies that are inherent in how networks actually communicate (Kasongo & Sun, 2020). In reality, intrusions rarely manifest as isolated anomalies. A Distributed Denial of Service (DDoS) attack, for example, involves sustained coordinated packet transmission over time; a probing attack systematically scans ports in sequential bursts. When models cannot represent these temporal patterns, the consequences are measurable: elevated false

positive rates, reduced detection accuracy, and degraded performance on precisely the attack types that are most dangerous.

High false alarm rates carry their own operational costs. When a detection system flags too many benign activities as malicious, security personnel experience alert fatigue, investigations consume unnecessary resources, and trust in the system erodes. Equally damaging is the converse problem — false negatives, where genuine attacks slip through undetected, exposing organisations to breaches that can be devastating. Striking the right balance between sensitivity and specificity remains an unresolved challenge. Ogunleye and Owolabi (2021) observed that the Nigerian cybersecurity environment faces compounding difficulties, including limited access to labelled training data representative of local network environments and insufficient computational infrastructure to support sophisticated detection models at scale.

This study addresses these problems directly. By designing and implementing an intrusion detection system grounded in LSTM networks, the research aims to capture the temporal structure of network traffic in a way that conventional classifiers cannot. Using multiple benchmark datasets and rigorous evaluation protocols, the study works toward a detection framework that is both accurate and generalisable — one capable of identifying known and novel attack patterns alike, and contributing meaningfully to network security practice in Nigeria and beyond.

1.3 Research Questions

The following research questions guide the investigation:

1. How can Long Short-Term Memory networks be effectively designed and implemented to model temporal dependencies in network traffic data for intrusion detection purposes?
2. What preprocessing techniques are most appropriate for transforming raw network traffic datasets into formats suitable for training deep learning models, specifically addressing categorical encoding, numerical normalisation, and class imbalance issues?

3. How does the performance of the proposed LSTM-based intrusion detection system, measured in terms of accuracy, precision, recall, and F1-score, compare with existing machine learning approaches documented in the literature?
4. What architectural configurations of LSTM networks — including the number of layers, units per layer, dropout rates, and activation functions — yield optimal detection performance on benchmark datasets including NSL-KDD, CICIDS2017, and UNSW-NB15?
5. To what extent can the proposed deep learning intrusion detection system generalise across different network traffic datasets, demonstrating robustness to variations in network environments and attack distributions?

1.4 Research Aim and Objectives

1.4.1 Research Aim

The overall aim of this research project is to design and implement a network based IDS by using a Long Short-Term Memory (LSTM) deep learning network that classify network traffic into normal or malicious based on temporal dependencies within the traffic, and that use benchmark datasets for detection accuracy.

1.4.2 Research Aim and Objectives

In this study, the following research objectives are set to achieve the above stated aim: Review and investigate the capabilities of the traditional, machine learning and deep learning IDS approaches with limitations and researches to be addressed.

Pre-process and prepare three popular network IDS benchmark datasets (NSL-KDD, CICIDS2017 and UNSW-NB15) using a cleansing, encoding, normalization, splitting mechanism of data into training and testing data set. Develop and implement an LSTM based intrusion detection system using deep learning techniques by selecting appropriate network

architecture and optimal model training/evaluation parameters. Evaluate and test the performance of the proposed LSTM based intrusion detection system using classification evaluation metrics and compare it with conventional machine learning classifiers.

1.5 Hypotheses

Null Hypothesis(H_0) :

Proposed LSTM based network intrusion detection system and conventional machine learning classifiers show no statistically significant difference in network intrusion detection capabilities.

Alternative Hypothesis(H_1) :

The proposed LSTM based network intrusion detection system shows a statistically significant superiority in network intrusion detection capabilities over the conventional machine learning classifiers in terms of accuracy, precision, recall and F1-score.

1.6 Significance of The Study

This research project will provide many benefits for Nigeria academia, industry and governmental institutions. Academics would greatly benefit as this research adds to the growing literature of using deep learning for security, and empirically supports using LSTM network for IDS with its corresponding methodology useful for researchers and graduate students working on the subject of network security, a new area of research in Nigeria academia.

Industry users like security analysts would greatly benefit because in Nigeria, we face a lot of security problems in the networks and with this developed intrusion detection tool, both novel and old network attacks could be detected efficiently with lesser training. This will ameliorate some security implementation issues currently in Nigeria (Akinwale and Adeyemi, 2020) by using cost-effective open-source tools such as Python, TensorFlow and Keras over expensive proprietary security products. Global benefit of this study is of immense importance as failure of many current systems to detect multistage modern attacks by properly utilizing the time dependencies information from the network traffic has been a great issue, which may be resolved by this research resulting in the reduction of many malicious events happening at networks. The outcome of this research may guide Nigerian policy makers in formulating robust network security policies.

1.7 Scope and Limitations

This research work only pertains to designing and implementation of a network-based intrusion detection system using a Long Short-Term Memory deep learning network which identifies network traffic as normal or malicious by classification.

This study was conducted using three public benchmark network IDS dataset, which are the NSL-KDD, CICIDS2017 and UNSW-NB15. All three datasets contain labeled network traffic with both normal data and different attack classes such as probe, DoS, U2R and R2L. The study used the Python programming language and its relevant deep learning libraries such as TensorFlow, Keras and PyTorch along with important data processing libraries such as Numpy, Pandas, Scikit-learn, Matplotlib and Seaborn. Diagrams, network architectures and flow charts were designed using Lucidchart, Draw.io and Microsoft Visio respectively, while versioning was carried out using Git. The research work is not without its limitations. The benchmark datasets used in this research study were simulated in a laboratory environment and are not necessarily reflective of Nigerian network traffic, thus serving as representatives of different forms of network attack classes. The NSL-KDD dataset which is a revised KDD Cup 1999 IDS dataset uses base data simulation that is outdated.

The experiments were conducted in a controlled environment, hence; the result obtained from laboratory tests could differ significantly from a real-world network system due to high traffic rate and network variability. Deep learning approaches beyond the LSTM network such as convolutional neural networks (CNNs) and Transformer models were not experimented with. It may result in better performance. Moreover, extensive hyper-parameter search was limited due to resource constraints. Deployment and evaluation in a live network was not within the scope of this study.

1.8 Definitions

IDS: It stands for network Intrusion Detection System which is an electronic device or software application that monitors a network for malicious activity or policy violations.

LSTM: A recurrent neural network (RNN) architecture which is specialized to learning long-term dependencies in time series data; it mitigates the problem of vanishing gradients found in traditional RNNs.

Deep Learning: A subset of machine learning approaches which use multi-layered artificial neural networks for the purpose of information processing.

TensorFlow: An open-source library developed by Google for numerical computation using data flow graphs that is primarily used for deep learning.

Keras: It is a high level API used to simplify and abstract away many complex operations associated with training deep neural networks which can be used on top of most other popular DL frameworks.

PyTorch: An open-source deep learning framework developed by Facebook, used widely by researchers due to its flexibility and ease of development of deep learning models.

NSL-KDD dataset: A hybrid approach of KDD 99. NSL-KDD dataset overcomes the issue of Redundant Record and Invalid Record which were in KDD Cup 99.

CICIDS2017 dataset: It is a contemporary dataset developed by the Canadian Institute for Cybersecurity, which includes a substantial volume of different types of attacks.

UNSW-NB15 dataset: An Intrusion Detection System (IDS) dataset which contains a hybrid of both benign and malicious traffic records obtained from Australian Centre for Cyber Security.

Accuracy: A metric used to evaluate the performance of a classification model by quantifying the percentage of correctly predicted instances over the total number of predictions.

Precision: It measures how many positive cases predicted by the model actually turn out to be positive, i.e. Out of all the negative predictions that are made, how many are actually negative.

Recall: A metric used to evaluate classification performance, it represents the proportion of actual positive instances that were correctly classified by the model.

F1-score: It is a metric that seeks to balance precision and recall and is commonly used when evaluating classification performance, specifically when the dataset is imbalanced.

Confusion Matrix: It is a table of the performance of a classification algorithm; the table itself includes counts of true positives, true negatives, false positives, and false negatives.

1.9 Structure of The Study

This research consists of five chapters. Chapter 1 deals with the problem of network security, establishes the basis for the proposed research work while other subsequent chapters provide details of literature review, methodology, experimental analysis of the work, and outcomes of the research.

Chapter One includes background, problem statement, research questions, research goal and objectives, hypotheses, significance, scope and limitation of the study, definitions, and a structure of the research.

In Chapter Two, a literature survey is carried out for intrusion detection systems from the era of rule-based methods until recent deep learning techniques. An assessment of existing experimental studies performed on LSTM based IDS is provided and existing research gaps are identified. The scope of the study is clearly placed in context to the relevant research literature available.

In Chapter Three, research methodology is explained in detail. It describes research design, system design, datasets, data pre-processing, the LSTM model design, tools and techniques employed and the evaluation metrics for the research. The entire steps for implementing the research are explained.

In Chapter Four, the process of implementation is described and the experimental results are discussed. It presents details on model training, experimental setup, result of evaluation experiments, analysis on confusion matrix, and detailed discussion on the results regarding the hypothesis and research questions.

In Chapter Five, the study is summarized. This section discusses main findings of the study and the conclusions that are drawn and recommendations for the future study are put forward. Also, contributions to knowledge are clearly pointed out.

CHAPTER TWO: LITERATURE REVIEW

2.1 Introduction

This section situates the present study within the vast and evolving research space of intrusion detection systems. This domain has come a very long way, from humble theoretical origins in the late 1980s to the deep learning architectures of today, but many of its initial oppositions remain hotly debated and unresolved. This chapter therefore is not just an exhaustive survey, but an examination of the manner in which this field of research has progressed and an identification of the knowledge gaps that remain the most salient and pressing.

The review is structured to progressively build the case for the chosen methodological approach for this investigation. It starts with the theoretical underpinnings of network security and IDS development, establishing the necessary vocabulary and context. Then it presents the standard taxonomies of IDS architectures: signature-based, anomaly-based, and hybrid systems. The advent of the machine learning paradigm, its influence on the analytical capacity of the field, then receives attention. A close look is paid to the deep learning architecture, focusing specifically on Recurrent Neural Networks and the Long Short-Term Memory variation, which forms the basis of this study. Empirical studies, based on comprehensive overviews and detailed investigations of methodologies, datasets utilized, observed outcomes and limitations. Benchmark datasets: NSL-KDD, CICIDS2017, UNSW-NB15 are given detailed attention as knowledge of their construction, and their shortcomings is paramount for interpretation of findings derived from their application. The Nigerian and African research context is incorporated throughout not as an exercise to lend credibility or a localized flavor, but as a genuine characteristic of the problem at hand. At the end, research gaps are identified, and a position for this investigation within the field is elaborated.

This review broadly addresses research in the field but analytically focuses on aspects most relevant to deep learning or LSTM-based IDS systems, the use of benchmark datasets for comparative evaluation, and the nature of empirical evidence reported. Efforts are made to balance international and African scholarship, with the limitations of relevant studies honestly presented.

2.2 Foundations of Intrusion Detection Systems

2.2.1 The Concept and Definition of an Intrusion Detection System

An Intrusion Detection System can be broadly defined as any system or combination of hardware and software components capable of monitoring and analyzing the activity within a computer or a network of computers, and generating an alarm upon detection of an anomaly. Early academic contributions on this front that has proven enduring include the real time IDS model, proposed by Dorothy Denning (1987), which was grounded on statistical analysis of system audit logs. Denning's original thesis proposed a deceptively straightforward model: Instead of identifying specific patterns indicative of an intrusion (which are obviously innumerable), one could characterize normal behavior and raise an alert for deviations from that profile. This idea – switching the detection focus from following rules to finding deviations from normality – remains the conceptual foundation to modern systems.

Over the years, this definition has expanded and refined. Modern IDS now integrate the notions of signature matching, protocol analysis, statistical profiling and combinations thereof with anomaly detection. The division between network IDS (which monitors traffic on the network) and host-based IDS (which analyzes activity on individual machines) has become a standard taxonomic division. Newer taxonomies now encompass hybrid or distributed IDS which combine both or data from multiple sources. What unifies all of these is the core purpose: early detection of security-relevant events so that a response can be enacted.

The importance of IDS within the modern network security framework can hardly be overstated. As Nigerian, African and worldwide organizations have increasingly come to rely on information systems accessible through networks, the surface areas exposed to attack has naturally increased. The magnitude of this problem for Nigerian organizations alone was documented in Akinwale and Adeyemi (2020), who reported billions in estimated annual losses and cited the absence of adaptive detection systems as an underlying cause for system vulnerability. Nigerian banks, which are undergoing massive digitalization, have also become a primary target for cybercriminals as transaction systems, customer databases, and interbank communication infrastructure represent high-value targets for those who seek to disrupt services or exploit

information. In the telecommunications industry, large volumes of highly diverse traffic create immense challenges for a rule-based security paradigm.

It is crucial to understand the IDS in the context of the overall defence-in-depth approach; it is not a self-sufficient defense. The value of the IDS lies in its ability to inform a response. The present study considers primarily the problem of detection, but its ultimate impact is realized in conjunction with incident response, forensics, and system remediation.

2.2.2 Historical Evolution of Intrusion Detection

The history of Intrusion Detection as an academic field can be traced back to Denning's (1987) paper, although it gained practical relevance in the 1990s with increasing internet use and public knowledge of intrusions. The DARPA-sponsored Lincoln Laboratory research program, initiated in the mid to late 1990s, resulted in the first real large-scale datasets to be used for the evaluation of IDS and led to benchmark testing becoming a community standard. This data, the KDD Cup 1999 dataset and subsequent versions, formed the accepted dataset for IDS evaluation for the next decade before flaws became evident enough that its successors were published.

The initial generation of commercial IDS software, such as Snort (Roesch, 1999) and ISS RealSecure, used the signature based approach to intrusion detection. Signature based IDS require a continuously updated list of known attack signatures, and raise an alarm when incoming data matches any pattern on this list. This signature based method works effectively for known attacks, with the practical advantage of having very low false positive rates when attempting to detect known attack types. This approach was particularly well suited to the threat landscape of the late 1990s and early 2000s, when the repertoire of common attacks was reasonably well known and the attack paradigm did not evolve rapidly.

However, limitations with signature based intrusion detection became apparent with the growth of polymorphic malware, and the increasing prominence of zero-day exploits. The latter meant there was a blind spot to this method since an attack occurred before it was known and its signature added to any detection database. It became evident that the vast majority of enterprise traffic on the internet makes manual maintenance of large signature files impractical. From these observations, there was a move towards anomaly based intrusion detection and eventually, data

driven methods to detection utilizing machine learning, to remove the requirement of predefined features in each attack.

Machine learning methods were predominant between the years 2005-2015. Algorithms such as SVM, decision trees, random forests and naive Bayes have been applied with success to IDS problems. With the growth in deep learning over the last five years, more sophisticated methods like convolutional networks, recurrent networks, and autoencoders, are now beginning to be widely implemented and applied to intrusion detection tasks. This research aims to focus on deep learning in the form of a Long Short-Term Memory, and apply it to learning the sequential nature of network attack types.

2.3 Types of Intrusion Detection Systems

2.3.1 Network Intrusion Detection System (NIDS)

Network based intrusion detection systems are used to observe traffic at multiple locations within a network, at the edge or internally at junctions and junctions and send these observations for analysis. These sensors gather and analyse packets passing through them, and compare this against defined criteria to determine any malicious activity. This architecture is particularly attractive as it allows for a single sensor to observe all traffic through that segment, without needing to install software on individual hosts, and is relatively easy to implement.

The advantages of NIDS systems relate to its position within the network. Because a sensor is placed within the network's core structure, it can easily monitor traffic from hundreds or thousands of devices within that network. NIDS are ideal for monitoring network-based attacks such as scans, flooding or protocol abuse. Commercial NIDS systems such as Snort (Roesch, 1999) and Suricata have accumulated rule sets, and support from a user community.

The limitations of NIDS can be easily identified as the volume and encrypting nature of data being passed around. Network traffic is now overwhelmingly encrypted, meaning any attack within this channel is completely undetectable to a packet inspection tool. High-speed traffic may also overwhelm NIDS, forcing compromises in network monitoring such as only observing a percentage of packets. It is also difficult for NIDS to trace attacks that span numerous sessions/connections.

2.3.2 Host-Based Intrusion Detection Systems (HIDS)

A Host-Based Intrusion Detection System is installed on a specific machine and looks at the telemetry on that host such as system calls, file access patterns, process behavior and user logins etc. Since a NIDS is an external observer and a HIDS is an internal observer, HIDS can detect the things that NIDS can't: privilege escalation on the local system, modifications to files and processes etc.. This makes HIDS valuable against those attacks that would not otherwise show up on the network traffic.

With the host-based nature, a degree of awareness can be attributed to the detection system. For example, the connection observed on the network could be associated with the actual process it was created by and which user account that process is running as. Any modifications that have been made to the configuration files since a compromise was last suspected could be compared against this. The correlation between a strange network connection and modifications that the program that initiated the connection has made to the system makes a potentially false positive alert much more useful. This makes HIDS ideal for those systems susceptible to targeted attacks. Examples include governmental systems, financial systems, and research network infrastructure.

There are significant issues regarding HIDS deployment. Managing a HIDS client on each system in large organizations makes deployment and maintenance a large, resource intensive task. Performance overhead on the host is created, and this can be a real problem when systems need high performance, as the more detailed and intrusive the analysis the more resource hungry it becomes. A major undertaking in getting alerts to make sense comes when comparing data from hundreds of heterogeneous operating systems and hardware setups. For this reason HIDS deployment is rarely universal but instead occurs selectively to bolster NIDS.

2.3.3 Hybrid Intrusion Detection Systems

Hybrid IDS combines network-level and host-level observation, and even multiple detection methodologies at the same time. It does this in order to make use of the strengths and mitigate

the weaknesses of both HIDS and NIDS. Typical architectures would send network packet data and host event data to a single location where correlation can be performed to combine different inputs. For example, an alert from a NIDS regarding unexpected network traffic could be combined with HIDS data on which program opened the network connection, the user which initiated the connection, and recent modifications that program has made to the system. The resulting detection would be much more specific and therefore more accurate than from a single sensor alone.

A hybrid architecture may also utilize multiple types of detection on top of the basic hybrid of HIDS and NIDS, such as signature-based and anomaly-based methods. The work by Kim et al. (2014) specifically studied combining two detection methodologies - namely the combination of LSTM layers and convolutional layers. The LSTM's ability to understand temporal relationships within packet data and the convolutional ability to perform spatial learning with regard to packet features resulted in a model that was able to score 98.2% on the NSL-KDD dataset, far more effectively than systems which did not utilize a hybrid approach. Their important conclusion is that two analytically different features are present in a single data stream, which if processed by a complementary architecture can yield a synergistic result that outperforms constituent models on their own.

As with most things in life, there are trade-offs, and with hybrid architectures this is complexity. Any further layer added to a system increases its overhead in terms of maintenance and computational requirement. The increased demands put on a computer that the paper by Kim et al. (2014) demonstrated are typical of hybrid architectures may not be feasible for institutions with limitations regarding available computer resources; specifically, it can be inferred that in many Nigerian institutions not classified as financial institutions or major telecom networks the simpler single architecture HIDS or NIDS may prove to be the best available option in terms of deployability versus capability.

2.4 Machine Learning Approaches to Intrusion Detection

2.4.1 The Paradigm Shift to Data-Driven Detection

A real conceptual shift to using machine learning for intrusion detection is demonstrated by the transition from signature based methods to data driven ones. Signature based intrusion detection systems coded explicit rules based on expert knowledge. As such they were always backward-looking and can only detect threats that were already known to the designers of the intrusion detection system. Machine learning instead has given intrusion detection systems the ability to learn from data themselves. The designer of the intrusion detection system did not have to specifically write detection logic for each potential attack (Goodfellow et al., 2016). The model would simply look for statistical regularities within data that were characteristic of an attack, and by extrapolating these findings it would in principle detect novel attacks that were not yet documented or known to anyone.

While these findings have become reality, it's to a somewhat limited extent. ML classifiers trained on IDS data sets can indeed produce superior results in testing and experimentation compared to static rule-based systems when detecting unfamiliar threats; the overall promise has been substantial and, in its own way, well-founded. However, one fundamental issue which has increasingly become clear is that standard ML considers observations in an independent fashion. When examining data within the intrusion detection system, whether it be network connections, flows, or packets; that data is observed devoid of all context and the temporal sequence in which the events occurred is disregarded. Although mathematically easier for an ML classifier to evaluate, this is not representative of what most modern intrusions look like. Attacks can often only be described through their temporal context over time.

2.4.2 Traditional Classifiers: Decision Trees, SVMs, and Random Forests

One of the few truly methodical comparative experiments for traditional machine learning classifiers for IDS was reported by Othman et al. (2018). In this work, decision trees, Naive Bayes classifiers and Support Vector Machines (SVMs) were tested on the NSL-KDD dataset. To ensure a level playing field between all the classifiers tested, the same feature preprocessing pipeline was used; the same experimental evaluation setup (with regard to test and training split)

was used; the same metrics were collected. While decision trees achieved the highest accuracy overall – taking advantage of the ability to create non-linear decision boundaries without the kernel engineering required by SVMs. The Naive Bayes approach was effective at binary classification but fell dramatically off for multi-class problems, since its independence assumption became more untenable with the addition of more classes. The single most critical conclusion reached by Othman et al. (2018) was neither the accuracy ranking of models nor any per-class detection figures. This critical conclusion was that all of the above models performed dramatically worse on any given attack type if that particular attack type had fewer examples than all others in the training partition. The U2R and R2L attacks (which are two of the most dangerous IDS attacks as they involve elevating privileges, and external to internal system access respectively) appeared so few times in the training set that the models simply could not learn to recognize them; this weakness was disguised by the high overall detection rate.

This is not just a single, isolated conclusion either. In fact, the overall sentiment in the machine learning IDS field is that detection rates are very high for a few common types of attacks (DoS and Probe) while remaining very low for other attack types. The assumption in the machine learning IDS field has been that while some attacks may be better understood and detected than others, on the whole this is not an issue as there are simply more examples of the former in the training set. This is of significant concern for security applications. An IDS that can detect, for example, a large, brute force denial of service attack, but fails to recognize an internal intrusion involving escalating a user to root privileges is of only limited use to an organization; the user-to-root attack is arguably more dangerous.

Tama et al. (2019) evaluated the problem from a different angle: in an effort to improve on single classifiers, they evaluated various ensemble techniques. Random Forest (an ensemble of decision trees, where the features are randomly sampled for each tree, and a prediction is generated through a majority vote) obtained notably higher accuracy and minority class recall than any of the individual classifiers. Ensemble techniques work for the high variance decision tree by averaging out each tree's individual errors, and this helps in detecting classes with fewer samples more efficiently. Tama et al. (2019) also discovered that using feature selection to identify only the most informative features for a given attack classification could dramatically improve the

accuracy and reduce computational overhead. An individual attack using features that do not relate to the attack provide no benefit but degrades the accuracy more subtly.

Salo et al. (2020) synthesized findings from 10 years (2010-2019) of data mining-based intrusion detection research across a number of datasets and algorithms. The same trend as established by individual studies was discovered in their meta-analysis. Firstly, ensembles, specifically Random Forests, dominated the highest accuracy figures. Secondly, feature selection significantly boosted the performance and speed. Thirdly, The NSL-KDD dataset was so pervasive it showed that reported IDS accuracy figures only reflected accuracy of IDS against a single, simulated dataset, rather than showing realistic IDS effectiveness. Salo et al. (2020) was particularly critical of the class imbalance issue and how this masked detection figures which reflected simple majority class accuracy rather than real detection accuracy. A system that returns a true positive rate of 96% on a dataset containing 96% benign traffic and 4% attack traffic may simply be the product of correctly classifying the non-attack traffic.

2.4.3 Feature Engineering Difficulties in ML-based IDS

One of the critical, and more subtle, issues of ML-based IDS is the challenge of feature engineering. The nature of network data is that it is provided in a raw, packet-based format. This needs to be aggregated in to a form usable by any conventional machine learning classifier, and thus any given set of features must be generated from the raw data stream in some consistent manner. The method of generation (for example, how many samples should make up each sliding window, which packet features should be used) have a very significant impact on what the model can learn. This requires a substantial domain knowledge base, and tends to become increasingly brittle as threat methods evolve. In fact, an attack may exploit a protocol feature that has been left out of the engineered feature set; a model will have no ability to classify this attack. Although, it is worth noting that deep learning has provided one way around the fragility of feature engineering in this domain by learning feature representations automatically, it does suffer from computational and training data challenges, in addition to losing interpretability. Kasongo and Sun (2020), though, still demonstrated that a carefully selected set of raw features could produce superior performance for their feedforward neural network.

2.5 Deep learning for intrusion detection

2.5.1 The arguments for deep learning in IDS over traditional machine learning

The arguments for using deep learning for intrusion detection can be grouped into two conceptually overlapping categories. The first is representational: neural networks with many layers of non-linear transformation can approximate any arbitrarily complex function from network traffic feature to attack category, including many functions that could never be parametrised by a hand-engineered set of features (LeCun et al., 2015). The second argument is one of temporal structure: certain network architectures, namely recurrent neural networks (RNNs) and long-short term memory (LSTM) variants, were designed explicitly to process sequence data, meaning that they can inherently learn temporal dependencies where they exist in the data. Conventional machine learning classifiers have no such ability.

The empirical literature largely supports both these arguments: when deep learning models are tested against their machine learning counterparts using the same datasets and the same testing protocols, deep learning models consistently exhibit higher accuracy and better minority class recall (especially for more complex attack taxonomies). The magnitude of the advantage varies from study to study, architecture to architecture, and testing protocol to testing protocol, but the trend is clear. However, the disadvantages remain consistent too: deep learning models require more computational resources during training, are more prone to over-fitting in smaller datasets and are harder to interpret than tree based models which are more visualisable. This means the appropriate balance of advantages and disadvantages depends on the application; for this study which prioritises detection of multiple attack categories through high accuracy, the argument appears to favour deep learning.

2.5.2 Convolutional neural networks for IDS

Convolutional neural networks (CNNs), first developed in computer vision and now applied to many pattern recognition tasks, were adopted by the intrusion detection community with the treating of network traffic feature vectors as structured inputs which could benefit from convolutional filtering. The main difference between CNN layers and standard fully-connected layers is that they only apply filters over small, localised regions of the input feature vector, thus

restricting the computation and weighting parameters that will be used, and enabling the detection of localised patterns in the input space. These local feature patterns often correspond to co-occurring features in network traffic data, enabling their detection and identification of them as discriminators between attack classes.

One of the first works applying deep learning architectures to the intrusion detection problem were by Javaid et al. (2015). In their study they trained stacked autoencoders on network traffic to compress representations, and use reconstruction error as an anomaly score. An autoencoder trained on normal traffic learns to reconstruct normal traffic correctly; anything other than normal is flagged as anomalous. This has the significant advantage that it only requires normal traffic in order to be trained; attack examples need not be present in training. However, the issue raised by Javaid et al. (2015) that only binary classification (normal versus anomalous) is possible with such an architecture renders it of limited use in a practical IDS application. This would require, for instance, an attack alert which classifies what type of anomaly has occurred rather than one stating merely "This traffic is anomalous."

Pushing on this concept further, Gueriani et al. (2024) implemented and tested a combined CNN and LSTM hybrid network for intrusion detection on the NSL-KDD and CICIDS2017 datasets. The CNN components processed their inputs (network traffic feature representation) to extract local spatial patterns (co-occurrences of features), while the LSTM layers processed these extracted features in sequence to capture the temporal dependencies which were crucial to intrusion detection. Combined, these captured different aspects of the input space, enabling them to achieve >99% accuracy on NSL-KDD and excellent minority class recall on the CICIDS2017 dataset. The major limitation pointed out by Gueriani et al. (2024) was that this hybrid architecture suffered from significant computational cost over the simple CNN or LSTM alone, potentially rendering them unsuitable for edge implementation, and was not tested in streaming/real-time applications.

2.5.3 Recurrent networks and the advantage of temporal modelling

RNNs process sequence data by maintaining a state vector, which at any given point in the sequence is a function of the input and the preceding state. This inherent structure allows the network to integrate context from previous inputs, meaning RNNs are much better at capturing

the long-range, sequential dependencies in data such as time-series traffic than their feed-forward counterparts which assume independence between inputs. Such long-range dependencies are essential in IDS, because a D DDoS attack can only be identified as an attack through its signature pattern in packet arrival times and payload sizes over many hundreds, if not thousands, of packets. This pattern can be learned by RNNs, and not by feed-forward networks.

The one drawback for practical applications of RNNs is the vanishing gradient problem, in which gradients are multiplicatively reduced at every time step when performing backpropagation through time across lengthy sequences. As a result, RNNs are notoriously bad at capturing temporal dependencies across long ranges, and a key issue in intrusion detection where these long-range patterns represent the bulk of the attack signatures.

2.6 Long Short-Term Memory (LSTM) Networks

2.6.1 Architecture and Gating Mechanism

Long Short-Term Memory (LSTM) networks were proposed by Hochreiter and Schmidhuber (1997) as an architectural solution to the vanishing gradient problem. The key innovation was the memory cell - a recurrent network unit whose value may be held over time and protected against deletion and decay of old memories to prevent the exponential degradation found in basic RNNs. The contents of the memory cell are modified and regulated by three learned gating functions: input gate, forget gate and output gate.

The input gate controls the amount to which the cell state should be updated in the present time step, using a sigmoid activation over a combination of current input and prior hidden state to determine a value between 0 and 1 which scales the proposed update to the cell state. The forget gate controls the extent to which the old cell state value is kept and discarded. The combined effect of input and forget gates is that information may be retained indefinitely if needed by ensuring that a large proportion of it is remembered and so, as a side effect, unwanted data is not stored indefinitely, only that which is useful to the current task. The output gate controls the value of the hidden state that will be output for the current time step.

Sherstinsky (2020) provided a rigorous mathematical derivation of LSTM dynamics by characterising the gating mechanism as a learned, input-dependent memory that makes cell state capable of carrying information over long temporal distances as opposed to RNN memory which is characteristically subject to exponential decay over time. LSTMs do not architecturally limit the memory length but instead it is a value learned from the training data-the duration of time that each piece of data is kept relies on the relevance to the target task, and in an IDS setting it is conceivable that LSTMs could learn to hold information about early probes such as initial recon packets over a long period until that information can be identified as a part of an attack.

Goodfellow et al. (2016) located LSTMs in a hierarchy of architectures and proposed that they solve the problem of vanishing gradients by allowing backpropagated signals, through learned input-dependent gating mechanisms acting as shortcuts, to travel through the neural network without exponential attenuation over time. The temporal signature of attacks which may have a considerable temporal spread, and that which present the most challenges (especially the User-to-Root and Remote-to-Local intrusions), can therefore be considered to be preserved as long-range information and it would be of structural benefit for an IDS to exploit this.

2.6.2 Why LSTM Is Optimal for IDS

The effectiveness of LSTMs in an IDS setting can be seen from their match with the nature of network intrusions which are predominantly temporal phenomena. The sequence of actions that make up an attack can have a temporal extent that is difficult to quantify by treating each discrete packet/connection independently; port scans are a series of packet probes to each consecutive port, brute force authentication is a series of failed login attempts, a multi-stage intrusion could involve initial network scanning followed by an attempt at system penetration, the subsequent lateral movement across the internal network and then data theft. The effectiveness of the LSTM is the ability to learn the temporal patterns underlying such occurrences while basic, non-recurrent and even standard recurrent networks would be structurally incapable of doing so; as stated by LeCun et al. (2015), the representational capacity of deep learning architectures relies on the capacity to discover hierarchical feature abstraction, but in an IDS setting it relies on the capacity of those features to represent information over the sequential dimension of the data, a property that the LSTMs are specifically designed to harness by maintaining an internal

memory that is constantly updated based on incoming data to represent the history of the data so far. This multi-dimensional feature extraction combined with an inherent robustness against vanishing gradients gives the LSTM the structural advantage needed to capture even complex and temporally diverse attack scenarios.

2.7 Review of Empirical Studies

2.7.1 International Studies

Laghrissi et al. (2021), publishing in the *Journal of Big Data*, conducted one of the more comprehensive single-system evaluations of LSTM-based intrusion detection. Their study employed the NSL-KDD dataset and constructed a multi-layer LSTM architecture trained to classify network traffic across all four NSL-KDD attack categories as well as normal traffic. The reported accuracy of 97.5%, combined with a false positive rate below 1%, placed the system among the top performers in the literature at the time of publication. What distinguished the Laghrissi et al. (2021) analysis from many comparable studies was the depth of their per-class performance reporting: rather than relying on aggregate accuracy figures alone, they reported precision, recall, and F1-score for each individual class, revealing that the LSTM maintained strong recall even on the U2R and R2L categories that consistently challenge conventional classifiers. The authors attributed this result directly to the LSTM's temporal modelling capability, arguing that these attack types — which characteristically unfold across multiple sequential events — are precisely the ones most likely to benefit from architectures designed for sequential data. The primary limitation was a failure to test cross-dataset generalisability; the NSL-KDD-exclusive evaluation left open the question of whether the strong results would replicate on more contemporary datasets with different traffic characteristics and attack distributions.

Rehman et al. (2025) contributed a wide-ranging systematic survey of machine learning and deep learning approaches to intrusion detection, synthesising findings from over one hundred individual studies published through the early 2020s. Their analysis provided a meta-level perspective on the field's development that individual empirical studies cannot offer. Several

important conclusions emerged. First, LSTM, and hybrid models (often LSTM with either CNN or an attention mechanism), were ubiquitously presented within the best-performing approaches for datasets, regardless of the task or evaluation framework. Second, results could rarely be reliably compared between works, due to different training/test split methods, data preprocessing techniques, and evaluation metrics, making it hard to conclude whether genuine performance differences or procedural artifacts are represented. Third, the over-reporting of results based on the NSL-KDD dataset relative to more contemporary datasets was a notable anomaly, suggesting the field's perception of progress may be artificially inflated by focus on a data set that does not accurately represent modern security threats. The report concluded with proposals of standardization in the evaluation framework, obligating evaluation across datasets, and closer observation of class imbalance as areas for improvement. These recommendations directly shaped the methodological design of the present study.

Al Jallad et al. (2019) took an unusual approach to intrusion detection by combining unsupervised clustering with supervised classification in a two-stage pipeline. In the first stage, K-means clustering organised traffic records into behavioural groups based on feature similarity, without reference to class labels. In the second stage, a supervised classifier was applied within each cluster, operating on a feature space that had been pre-organised by the clustering step. Evaluated on both NSL-KDD and UNSW-NB15, the pipeline showed improved detection rates for minority attack categories compared to classifiers applied directly to the unclustered data. The intuition behind this result is that clustering reduces the within-class variability that minority category classifiers must contend with, making the decision boundaries learned by the supervised component more reliable. The limitation, which the authors acknowledged, was that the two-stage pipeline added complexity and required careful calibration of the clustering parameters — parameters whose optimal values depended on dataset characteristics that might not generalise across deployment environments. The study's cross-dataset evaluation on NSL-KDD and UNSW-NB15 represented good practice, but the pipeline's deployment readiness remained uncertain.

Saurabh et al. (2022), writing in a context motivated by the expansion of Internet of Things infrastructure, extended LSTM-based intrusion detection to resource-constrained device environments where the performance characteristics of the standard enterprise network IDS

assumption do not apply. IoT networks present a distinctive set of detection challenges: device heterogeneity means that normal traffic profiles vary enormously across device types; the limited computational resources available for on-device processing constrain the architectural complexity of any embedded detection component; and the attack surface is vastly broader than conventional enterprise networks because IoT devices often run minimal operating systems with limited security hardening. Saurabh et al. (2022) addressed the detection problem by employing Bidirectional LSTM layers, which process each input sequence both in its natural forward order and in reverse, allowing the model to capture dependencies that run in both temporal directions. Applied to IoT-specific traffic datasets, their model achieved detection rates exceeding 95% for DoS and botnet attacks — the most common attack categories targeting IoT infrastructure. The acknowledged limitation was latency: bidirectional processing requires the complete sequence to be available before processing can begin, which conflicts with the real-time detection requirements of many deployment contexts. The study nonetheless demonstrated that the LSTM architecture's effectiveness is not limited to enterprise network environments; it generalises to specialised and constrained settings with appropriate adaptation.

Kasongo and Sun (2020), in a study published in *Computers & Security*, investigated the interaction between feature selection and deep learning performance on the CICIDS2017 dataset. Their central contribution was a demonstration that a feedforward neural network trained on a carefully selected feature subset — chosen through a combination of correlation analysis and mutual information — outperformed the same architecture trained on all available features. This finding challenged a naive expectation that adding more information should always improve model performance, and it has important practical implications: the 80 features available in CICIDS2017 are not all equally informative, and some actively harm performance by introducing noise or confounding the learning process. The detection accuracy of approximately 89% reported by Kasongo and Sun (2020) was meaningful but fell short of LSTM-based results on NSL-KDD. The authors were clear about why: their feedforward architecture, regardless of the quality of its features, treated each traffic observation independently and could not leverage the temporal structure that LSTM networks exploit. Their study is best understood not as a performance benchmark but as a methodological demonstration — showing that feature management remains important even within deep learning paradigms, and that architectural choices and feature choices interact in ways that deserve careful attention.

Kim et al. (2014) explored hybrid intrusion detection in a deep learning context by combining CNN and LSTM components within a single unified architecture. The CNN layers operated on raw network packet representations, using convolutional filters to extract local feature patterns from within individual packets or short packet windows. These spatially encoded representations were subsequently input to LSTM layers that modeled temporal variations in traffic flow. The intuition is intuitive; spatially encoded (via CNN) packet-level patterns, and temporally encoded (via LSTM) flow-level patterns contain complementary information, so a combined model is expected to outperform a single-model baseline. Indeed, empirically, the hybrid CNN-LSTM model performed much better than the standalone models with 98.2% on NSL-KDD (using the same evaluation protocol). They identified two drawbacks: First, combined network is computationally far more expensive than single network so that makes it impractical for real time deployment. Second, they evaluated only on NSL-KDD, so they questioned whether this was just coincidence due to evaluation datasets. Gueriani et al. (2024) expanded on this by showing that CNN-LSTM can attain above 99% accuracy on NSL-KDD, and perform well on CICIDS2017; CNN pre-processing provided minor gains on recall for minority attacks compared to only LSTM baseline. This experiment used two datasets instead of one but again there was no real time evaluation, and UNSW-NB15 is missing from the datasets.

2.7.2 Nigerian and African studies

Akinwale and Adeyemi (2020) presented some contextually important empirical evidence supporting the motivation for the current study. They analyze the size and nature of cybercrime in Nigeria, drawing on quantitative data from Nigerian financial institutions and government cybersecurity agencies alongside survey evidence from academia. They reported that financial organizations suffered continuous intrusions on their transaction processing and customer database systems. Government institutions faced politically motivated attacks alongside opportunistic theft, while most attacked entities did not have the in-house capability to detect, contain or investigate. The study's analytical contribution extended beyond documenting impact to identifying structural vulnerabilities. Nigerian organisations, Akinwale and Adeyemi (2020) argued, are disproportionately reliant on signature-based commercial IDS solutions sold by international vendors, creating a dependency on continuous signature updates that assumes technical infrastructure and vendor relationships that many Nigerian institutions cannot sustain.

The rapid evolution of the threat landscape means that the lag between the emergence of a new attack technique and the publication of a corresponding detection signature represents a period of systematic exposure — and that this exposure is longer and more consequential in environments where signature update infrastructure is unreliable.

Ogunleye and Owolabi (2021) examined the potential of artificial intelligence-based detection systems to address this structural vulnerability, making a case that rested on both technical and institutional grounds. The technical argument was straightforward: a deep learning model trained to detect intrusions from data, rather than from curated rules, does not require continuous expert maintenance in the same way that a signature-based system does. Once trained, the model's detection capability reflects the patterns present in its training data and does not degrade as new attack signatures emerge — though it may need periodic retraining as the traffic distribution drifts over time. The institutional argument was more nuanced: the adaptive, data-driven nature of deep learning models makes them more sustainable in resource-constrained environments, not because they are maintenance-free, but because the maintenance they require — periodic retraining on new data — is more compatible with the technical capacities of Nigerian organisations than the continuous expert curation required by signature-based systems. Ogunleye and Owolabi (2021) acknowledged that initial deployment of deep learning IDS requires significant computational investment and the availability of representative training data, both of which represent real barriers for smaller institutions. Their study was primarily conceptual rather than empirical, and the absence of a deployed system evaluation limits the strength of the conclusions that can be drawn from it.

Oladeji and Adelabu (2019), in a comparative study grounded in the African institutional context, evaluated several machine learning and deep learning architectures on publicly available IDS datasets, framing their analysis explicitly around the operational realities of African university networks. University networks in Nigeria and other African countries present a distinctive security profile: they support large numbers of users with diverse access needs and behaviour patterns, they operate on relatively modest infrastructure compared to enterprise networks in high-income countries, and they face both opportunistic attacks from external actors and security risks from students and staff within the network perimeter. Oladeji and Adelabu (2019) found that LSTM-based models outperformed other evaluated architectures on the recall

metric — the proportion of actual attacks that the model correctly identified — a finding with direct operational significance, since missed detections (false negatives) expose the network to harm in a way that false alarms (false positives) do not, though both impose operational costs. Their most significant contribution to the present study's framing, however, was methodological rather than empirical: Oladeji and Adelabu (2019) raised, explicitly and with appropriate persistence, the concern that training data derived from North American and Australian network environments is not representative of African network traffic. The applications predominant on African university networks, the infrastructure characteristics of the underlying connectivity, and the user behaviour patterns are sufficiently different from those captured in NSL-KDD, CICIDS2017, and UNSW-NB15 to warrant caution about the direct transferability of models trained exclusively on these datasets. This concern, while unresolved in the present study — which, like virtually all IDS research in Nigeria, relies on available international benchmark datasets for lack of locally collected alternatives — remains one of the most important open challenges in the field.

2.8 Benchmark Datasets in IDS Research

2.8.1 The NSL-KDD Dataset

The NSL-KDD dataset was introduced by Tavallae et al. (2009) as a corrected version of the KDD Cup 1999 dataset, which had by the late 2000s accumulated a substantial body of criticism. The original KDD data contained massive numbers of duplicate records — a consequence of the simulation methodology used to generate it — and this redundancy allowed models to achieve very high accuracy scores by memorising specific instances rather than learning generalisable patterns. Removing the duplicates and rebalancing the training and test partitions were the primary contributions of the NSL-KDD revision, and they substantially improved the dataset's utility as a generalisation benchmark.

In the NSL-KDD dataset there are 41 features in each record: these features are based on connection-level statistics (duration, protocol type, service, flag), content-based features (number of failed logins, number of root shell calls), and time-based statistics (number of connections to the same host within the previous 2 seconds). The attack taxonomy is defined within four main types: DoS, Probe, U2R, R2L. There are 125973 records within the training set, and 22544

records within the test set, where the test set actually contains a much larger number of minority class attacks (U2R and R2L) than the training set, as this intentionally makes the challenge more rewarding of real generalization rather than training set fitting.

Despite these improvements, NSL-KDD carries limitations that are now widely acknowledged in the literature. The underlying network traffic was simulated in a controlled laboratory environment in 1998, an era that predated wireless networking, cloud computing, streaming media, social media platforms, and the modern web as it exists today. The traffic patterns captured in the dataset bear little resemblance to contemporary enterprise network traffic, and the attack categories present — while historically important — do not include many of the threat types that dominate the current security landscape: ransomware, advanced persistent threats, supply chain attacks, and modern botnet activity are all absent. Rehman et al. (2025) identified the literature's continued reliance on NSL-KDD as a source of systematic optimism bias: models evaluated exclusively on this dataset may perform impressively against attacks that are in many senses already solved problems while exhibiting unknown performance against the threats organisations actually face today. The present study employs NSL-KDD alongside more recent datasets precisely to maintain comparability with the historical literature while guarding against the false confidence that NSL-KDD-only evaluation would produce.

2.8.2 The CICIDS2017 Dataset

The CICIDS2017 dataset, developed by Sharafaldin et al. (2018) at the Canadian Institute for Cybersecurity, was designed from the outset to address the currency problem that makes NSL-KDD an inadequate sole benchmark for contemporary IDS research. Rather than simulating traffic using synthetic generation tools, the CICIDS2017 team constructed a realistic network testbed populated with real operating systems running real applications with user behaviour profiles derived from empirical research on actual network usage patterns. Network traffic was captured over five consecutive days, with attack scenarios introduced at scheduled intervals to simulate the range of threats that contemporary enterprise networks face. The resulting dataset includes benign traffic as well as brute force attacks, Heartbleed exploitation, botnet activity, DoS and DDoS attacks, web attacks (SQL injection, XSS, and brute force), and infiltration scenarios involving both internal and external attackers.

Traffic was captured using CICFlowMeter, which extracted 80 bidirectional flow features from raw packet captures. These features include statistical summaries of inter-arrival times, packet lengths, flag counts, and flow durations, as well as features specific to the forward and backward directions of each flow. The resulting feature set is considerably richer than NSL-KDD's 41 features and better reflects the characteristics of modern network traffic, which is predominantly flow-based rather than packet-based in its most informative representation.

CICIDS2017's strengths are real and significant: the traffic is contemporary, the attack types are operationally relevant, and the feature set is more informative about modern network behaviour than any predecessor dataset. Its limitations are equally real. The class distribution is severely imbalanced — benign traffic accounts for the vast majority of records, and several attack categories are represented by only a few thousand instances each, making minority class detection particularly challenging. A subset of feature columns contains infinite or NaN values arising from division-by-zero errors in the flow feature extraction process, and these must be handled carefully during preprocessing to avoid introducing bias. The temporal structure of the dataset — with traffic captured on specific days with specific attack schedules — means that random splitting of records can introduce data leakage, where the model learns time-of-day or day-of-week correlates of attack categories rather than genuine attack signatures. These are not disqualifying limitations, but they require methodological care that the literature has not always exercised.

2.8.3 The UNSW-NB15 Dataset

The UNSW-NB15 dataset (created by Moustafa and Slay, 2015) of the Australian Centre for Cyber Security at the University of New South Wales falls somewhere in between NSL-KDD and CICIDS2017 in both age and design philosophy. It was generated using the IXIA PerfectStorm traffic generation tool, which produced a combination of real network traffic and synthetically generated attack traffic. The attack taxonomy spans nine categories — Fuzzers, Analysis, Backdoors, Denial of Service, Exploits, Generic, Reconnaissance, Shellcode, and Worms — representing a broader range of threat types than either NSL-KDD or CICIDS2017. The full dataset contains approximately 2.54 million records described by 49 features, making it one of the largest publicly available IDS benchmarks.

The diversity of UNSW-NB15's attack taxonomy is its most distinctive advantage. Categories such as Fuzzers, Shellcode, and Backdoors represent threat types that are entirely absent from NSL-KDD and only partially represented in CICIDS2017. A model evaluated across all nine UNSW-NB15 attack categories is tested against a broader range of intrusion techniques than any other single benchmark dataset provides, and the resulting performance profile is more informative about the model's genuine generalisability. The combination of real and synthetic traffic also provides a more varied feature distribution than purely simulated datasets, exposing the model to the kind of feature diversity it will encounter in real deployment environments.

Class imbalance is a challenge in UNSW-NB15 as in every other IDS benchmark — the Normal class accounts for the substantial majority of records. The synthetic traffic generation component means that some attack instances may not fully replicate the characteristics of real-world attacks; the IXIA-generated traffic, while capturing the structural features of attack categories, may lack the subtle irregularities and contextual variations that distinguish real intrusion traffic from its simulated equivalent. Nevertheless, UNSW-NB15 has established itself as a rigorous and widely cited benchmark, and its inclusion in the present study alongside NSL-KDD and CICIDS2017 significantly strengthens the generalisability claims of the proposed system.

2.9 Identified Research Gaps

A sustained reading of the literature reveals several gaps that persist despite decades of research activity. These are not merely theoretical inadequacies; they have practical consequences for the reliability and deployability of IDS systems. Each gap identified below directly informs a methodological decision made in the present study.

The first and most fundamental gap concerns temporal modelling. The overwhelming majority of machine learning-based IDS studies — and a surprising number of deep learning studies — treat network traffic records as independent observations, learning to classify each record based solely on its own feature values without reference to the context provided by preceding or subsequent records. This is a substantial mismatch between modelling assumption and empirical reality. Network attacks, with rare exceptions, are not isolated events. They are processes that unfold across time, leaving temporal signatures distributed across sequences of network events. A classifier that cannot represent these signatures cannot reliably detect the attack types that

depend on them. LSTM-based studies have demonstrated the value of temporal modelling, but many employ architecturally shallow models or do not systematically investigate how sequence length, layer depth, and architectural configuration affect the quality of temporal representations. A deeper and more systematic treatment of temporal modelling is needed.

The second gap is poor cross-dataset generalisation. The literature is heavily dominated by single-dataset evaluations, with NSL-KDD overrepresented to a degree that Salo et al. (2020) and Rehman et al. (2025) both identified as a source of systematic bias. Models evaluated on a single dataset may perform well because they have learned dataset-specific regularities — artefacts of the simulation process, idiosyncrasies of the traffic generation procedure, or correlates of the labelling methodology — rather than genuinely generalisable attack signatures. The failure to test across multiple datasets with different traffic characteristics, attack distributions, and feature spaces limits the field's ability to make credible claims about real-world performance. The present study addresses this gap by evaluating the proposed system across three datasets with substantially different characteristics.

The third gap involves class imbalance handling. Virtually every IDS dataset is imbalanced, and virtually every IDS paper acknowledges this. Yet many studies continue to report aggregate accuracy as their primary performance metric, a measure that is deeply misleading when one class dominates the distribution. A model that achieves 96% accuracy on a dataset where 95% of records are benign has learned nothing useful for intrusion detection. Minority class recall — the fraction of actual attacks that the model correctly identifies — is the metric that most directly reflects the system's operational value, and it is frequently either unreported or reported without contextualising it against the difficulty of the minority class detection problem. Better handling of class imbalance, through appropriate sampling strategies, loss weighting, and evaluation metrics, is essential for the field to make credible progress.

The fourth gap concerns overfitting in deep learning models. Deep neural networks have sufficient capacity to memorise their training data rather than learning generalisable patterns, and this risk is compounded in IDS by the relatively small size of many benchmark datasets relative to the complexity of the models applied to them. Regularisation through dropout, early stopping, and weight decay is widely employed, but inconsistently applied and inconsistently reported. The

literature rarely provides sufficient detail about regularisation choices to allow readers to assess whether reported performance figures reflect genuine generalisation or training-set memorisation. More rigorous and transparent treatment of overfitting risk is needed.

The fifth gap is the limited contextualisation of IDS research for Nigerian and African deployment environments. As Oladeji and Adelabu (2019) and Ogunleye and Owolabi (2021) have documented, the challenges of deploying effective intrusion detection in Nigerian institutional contexts are substantially different from those addressed by the North American and European research that dominates the literature. Benchmark datasets are derived from network environments with different infrastructure characteristics and user behaviour patterns from those typical of Nigerian institutions. Computational resources available for training and running detection models are more constrained. Ongoing technical support for signature-based systems is more difficult to maintain. Research that takes these realities seriously, rather than treating Nigerian deployment as a straightforward application of methods developed for different contexts, is needed and remains scarce.

2.10 Summary of the Literature Review

The literature surveyed in this chapter traces a coherent line of intellectual development from Denning's (1987) foundational anomaly detection model through the machine learning revolution to the deep learning architectures that currently define the frontier of IDS research. At each stage, the field's progress has been driven by a tension between what existing approaches could do well and what the nature of the intrusion detection problem actually required — between the operational simplicity of signature matching and its fundamental retrospectivity; between the adaptability of machine learning and its blindness to temporal structure; between the power of deep learning and the risks of overfitting and computational cost.

The theoretical case for LSTM-based intrusion detection is grounded in a precise alignment between architectural capability and problem structure. Network intrusions are temporal processes; LSTM networks are temporal sequence learners. The experimental results presented in this thesis and others like them (Laghrissi et al. 2021, Gueriani et al. 2024, Saurabh et al. 2022, Kim et al. 2014, Kasongo and Sun 2020 and the survey research in Rehman et al. 2025) suggest that there is such a correlation. The LSTM and hybrid LSTM methods have consistently

achieved high accuracy across each of the datasets used, and importantly on the minority attack groups that are actually significant to the defense system.

The three datasets used in this paper are NSL-KDD, CICIDS2017 and UNSW-NB15. Each of these datasets have unique characteristics making them unique on their own, but valuable together. NSL-KDD provides historical comparability with a large body of prior work; CICIDS2017 provides contemporary attack coverage with realistic traffic generation; UNSW-NB15 provides taxonomic breadth with a diverse nine-category attack taxonomy. Using all three guards against the dataset-specific overfitting that single-dataset evaluations invite.

The Nigerian context, grounded in the scholarship of Akinwale and Adeyemi (2020), Ogunleye and Owolabi (2021), and Oladeji and Adelabu (2019), establishes that the research problem addressed here is not merely a technical abstraction but a practical need with genuine consequences for institutions and individuals. The structural limitations of signature-based IDS in resource-constrained environments, and the potential of adaptive deep learning models to address those limitations, define a real-world motivation that goes beyond performance benchmarking.

The five gaps identified — in temporal modelling depth, cross-dataset generalisation, class imbalance handling, overfitting management, and contextual relevance — define precisely the space that the present study occupies. Chapter Three details the methodological choices through which the study addresses each of these gaps, from the sequence construction procedure that operationalises temporal modelling to the multi-dataset evaluation protocol that tests generalisation to the class weighting strategy that addresses imbalance.

CHAPTER 3: MATERIALS AND METHODS

3.1 Introduction

This chapter sets out the systematic procedures, materials, and analytical techniques that underpin the design and implementation of a deep learning intrusion detection system using Long Short-Term Memory networks. Rather than treating methodology as a formality, the choices documented here reflect deliberate decisions about how to handle data responsibly, construct a model that captures the temporal structure of network traffic, and evaluate outcomes in a manner that supports genuine comparisons with prior work. The importance of methodological transparency in deep learning research cannot be overstated: reproducibility depends on precisely the kind of procedural detail that this chapter provides (Chollet, 2017). The methodological foundations rest on established principles in experimental computer science and draw on recognised practices in deep learning system development (Goodfellow et al., 2016).

The overall approach is quantitative and experimental. Performance is measured empirically using standard classification metrics, and the experimental design is structured to isolate the contribution of the LSTM architecture to detection outcomes. The chapter proceeds through research design, materials and tools, the step-by-step experimental procedure, data analysis techniques, and ethical considerations.

3.2 Research Design

The study adopts an **experimental design** structured around the standard machine learning workflow for classification tasks. This choice is appropriate because it allows systematic manipulation of the independent variables — model architecture, hyperparameters, and preprocessing decisions — while measuring their effects on a well-defined dependent variable: intrusion detection performance. The experimental workflow is organised into five phases: data acquisition and exploratory analysis, preprocessing and feature engineering, model design and implementation, training and optimisation, and evaluation and validation (Kasongo & Sun, 2020).

A **comparative analysis** dimension is built into the design. The LSTM-based system is benchmarked against Random Forest, Support Vector Machines, and standard Recurrent Neural Networks, ensuring that performance claims are grounded in objective comparison rather than isolated observation. A **stratified cross-validation strategy** is employed to reduce the risk of overfitting and to verify that the model performs reliably across different subsets of the data. Critically, the design preserves the temporal ordering of network traffic sequences, since LSTM networks are architecturally built to exploit exactly this sequential structure (Hochreiter & Schmidhuber, 1997). All experimental runs are tracked using **Git**, ensuring that every code modification, configuration change, and result is traceable and recoverable.

3.3 Materials and Reagents

3.3.1 Datasets

Three benchmark datasets, each with a distinct profile of strengths and limitations, provide the empirical foundation for this study. Their combined use guards against the kind of dataset-specific overfitting that plagues single-dataset evaluations.

The **NSL-KDD dataset**, proposed by Tavallaee et al. (2009) as a corrected version of the KDD Cup 1999 data, remains the most widely cited benchmark in the IDS literature. Its primary contribution over its predecessor is the removal of duplicate records — a problem that allowed earlier models to achieve inflated accuracy scores by memorising specific instances. The training partition contains approximately 125,973 records and the test set 22,544, each described by 41 features covering connection-level attributes such as protocol type, service, and various statistical summaries. The attack taxonomy spans four categories: Denial of Service (DoS), Probe, User-to-Root (U2R), and Remote-to-Local (R2L). Despite its utility, NSL-KDD is based on simulated traffic from the late 1990s and does not reflect the network protocols, application traffic, or attack methodologies of the contemporary threat landscape.

The **CICIDS2017 dataset**, developed by Sharafaldin et al. (2018) at the Canadian Institute for Cybersecurity, addresses the currency problem head-on. Generated from a realistic testbed network running authentic user behaviour profiles, it captures both benign traffic and a range of modern attack types including brute force, heartbleed, botnet, and DDoS. CICFlowMeter was

used to extract 80 bidirectional flow features from over 2.8 million records collected across a five-day capture period. The dataset better reflects contemporary enterprise traffic than NSL-KDD, but its severe class imbalance — benign records vastly outnumber attack records in several categories — and the presence of infinite and NaN values in some feature columns require careful preprocessing.

The **UNSW-NB15 dataset**, developed at the Australian Centre for Cyber Security by Moustafa and Slay (2015), combines real network traffic with synthetically generated attack traffic produced by the IXIA PerfectStorm tool. It contains approximately 2.54 million records described by 49 features, spanning nine attack families: Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, and Worms. The breadth of this taxonomy exposes the model to threat types not represented in either NSL-KDD or CICIDS2017, strengthening the generalisability case for the proposed system.

3.3.2 Software Libraries and Frameworks

Python 3.9 serves as the implementation language, chosen for the unmatched depth of its data science and deep learning ecosystem. **TensorFlow 2.10**, accessed via the Keras API, is the primary deep learning framework: it provides high-level abstractions for building and training neural networks while supporting GPU acceleration through CUDA, which is essential for managing the computational demands of LSTM training on large datasets. **PyTorch 1.13** supplements TensorFlow for comparative experiments and for components requiring more granular control over the training loop.

NumPy handles the multi-dimensional array operations that are fundamental to deep learning computations. **Pandas** manages the loading, inspection, and transformation of the CSV-format dataset files. **Scikit-learn** contributes preprocessing pipelines, feature scaling utilities, and the evaluation functions — classification reports, confusion matrices, and ROC curves — used to quantify model performance. **Matplotlib** and **Seaborn** generate the visualisations — training history plots, confusion matrix heatmaps, performance comparison charts — through which results are communicated.

3.3.3 Hardware and Development Environment

Experiments were run on a workstation with an **Intel Core i7-12700H processor** (14 cores, 20 threads), **32 GB DDR5 RAM**, and an **NVIDIA GeForce RTX 3060 GPU** with 6 GB VRAM. The GPU was particularly important for training the model on the CICIDS2017 dataset, where the volume of records makes CPU-only training impractical within reasonable time constraints. The operating system was **Ubuntu 22.04 LTS**, which provides native CUDA support. Package management was handled through **Anaconda**, and **Jupyter Notebooks** served as the interactive development environment, facilitating iterative experimentation and inline documentation.

3.4 Equipment and Apparatus

3.4.1 Software Tools

System architecture diagrams and data flow charts were produced using **Microsoft Visio**, which facilitated the clear visual representation of module interactions within the IDS pipeline. **Lucidchart** supported collaborative design of the LSTM model architecture, capturing the layer structure, activation functions, and data flow through the network in a format accessible to non-specialist readers. **Draw.io** was used for the preprocessing pipeline and training workflow flowcharts that appear in the results chapter.

Git managed version control throughout the project. A GitHub repository tracked all code changes, preprocessing scripts, model configuration files, and experimental outputs, ensuring that the research record is complete and that any experimental run can be reconstructed from its logged configuration.

3.4.2 Data Storage and Management

The three datasets were stored on an external SSD (1 TB) to ensure read speeds adequate for the large file sizes involved. Metadata — feature descriptions, attack category mappings, and preprocessing parameter records — were managed using SQLite, which permitted efficient querying of dataset characteristics without loading full files into memory.

3.5 Experimental Procedure

3.5.1 Data Acquisition and Exploration

The three datasets were downloaded in CSV format from their respective official repositories: NSL-KDD from the University of New Brunswick, CICIDS2017 from the Canadian Institute for Cybersecurity, and UNSW-NB15 from the Australian Centre for Cyber Security. Each was subjected to exploratory data analysis (EDA) before any preprocessing was applied. Using Pandas for summary statistics and Matplotlib with Seaborn for visualisation, the EDA phase characterised feature distributions, identified missing and infinite values, confirmed class label distributions, and flagged potential redundancy between features. This step informed every subsequent preprocessing decision and is documented in full in the accompanying code repository.

3.5.2 Data Preprocessing Pipeline

The preprocessing pipeline was designed to transform raw, heterogeneous dataset files into temporally structured input sequences ready for LSTM training. Each step was applied consistently across all three datasets, with dataset-specific parameters where necessary.

Data Cleaning. Missing and infinite values — particularly prevalent in CICIDS2017 — were handled through mean imputation for continuous features and mode imputation for categorical ones. Duplicate records were identified and removed. These operations were performed before any scaling or encoding to avoid introducing bias into imputed values.

Categorical Encoding. Protocol type, service, and flag fields in NSL-KDD are categorical; they were transformed into binary representations via one-hot encoding, expanding the feature space but ensuring that ordinal assumptions were not incorrectly imposed on nominal variables.

Feature Scaling. All continuous features were normalised using Min-Max scaling, mapping values to the $[0, 1]$ range. This step matters particularly for LSTM networks because large differences in feature magnitude can destabilise gradients during backpropagation (Chollet, 2017). Scaling parameters — minimum and maximum values — were computed exclusively on the training set and applied to validation and test sets, preventing any form of data leakage.

Label Encoding. The target variable was encoded as integers: 0 for normal traffic and ascending integers for each attack category. For NSL-KDD specifically, the mapping was 1 for DoS, 2 for Probe, 3 for R2L, and 4 for U2R.

Sequence Construction. Temporal structuring is what distinguishes this preprocessing pipeline from those used with conventional classifiers. A sliding window of width 10 was applied across the ordered traffic records, constructing input sequences where each sequence contains 10 consecutive network connections. The label assigned to each sequence corresponds to the traffic class of its final time step. The window size of 10 was selected based on hyperparameter search results, which showed diminishing returns for longer windows given the feature characteristics of the datasets used.

Data Splitting. The full preprocessed dataset was divided into training (70%), validation (15%), and test (15%) partitions using stratified sampling, which preserves class proportions across all three splits. The validation set was used exclusively for hyperparameter tuning and early stopping decisions; the test set was held out until final evaluation.

3.5.3 LSTM Model Architecture Design

The model was constructed using the Keras Sequential API. The architecture reflects a deliberate balance between representational capacity and regularisation, motivated by the well-documented risk of overfitting in deep learning models trained on datasets of moderate size (Goodfellow et al., 2016).

The **Input Layer** accepts sequences of shape $(10, n)$, where n is the number of features after preprocessing: 41 for NSL-KDD, 80 for CICIDS2017, and 49 for UNSW-NB15.

The **First LSTM Layer** contains 128 units with tanh activation and sigmoid recurrent activation, configured to return the full output sequence (`return_sequences=True`) so that the subsequent LSTM layer receives temporal context at every step. A dropout rate of 0.2 is applied.

The **Second LSTM Layer** contains 64 units with identical activation settings but is configured to return only the final time step output (`return_sequences=False`), distilling the learned sequence representation into a fixed-length vector. Dropout of 0.2 is applied here as well.

A **Dense Hidden Layer** with 32 neurons and ReLU activation follows, incorporating L2 regularisation ($\lambda = 0.001$) to further penalise large weight values and discourage overfitting.

The **Output Layer** is a softmax-activated dense layer whose neuron count matches the number of target classes for the dataset in question, producing a probability distribution across categories.

The model is compiled with the **Adam optimiser** (initial learning rate 0.001), **categorical cross-entropy** as the loss function, and accuracy as the monitored metric. Adam was selected over alternatives such as SGD because its adaptive learning rate mechanism tends to converge faster and more reliably across the diverse dataset characteristics encountered in this study (Goodfellow et al., 2016).

3.5.4 Model Training and Optimisation

Training ran for up to 100 epochs with a batch size of 64, subject to early stopping with a patience of 10 epochs monitoring validation loss. Early stopping serves a dual purpose: it prevents overfitting by halting training before the model begins memorising training-set noise, and it reduces unnecessary computational expenditure on runs that have already converged.

Hyperparameter tuning was conducted via grid search over: LSTM layer count (1, 2, 3), units per layer (32, 64, 128, 256), dropout rate (0.1, 0.2, 0.3, 0.5), learning rate (0.01, 0.001, 0.0001), and batch size (32, 64, 128). Validation accuracy determined the optimal configuration.

Class imbalance — a persistent challenge in IDS datasets, and one that systematically disadvantages minority attack classes (Salo et al., 2020) — was addressed by computing inverse-frequency class weights and incorporating them into the training loss. This encouraged the model to attend to underrepresented categories rather than optimising primarily for the majority class. Model checkpointing saved weights at each epoch where validation loss improved, ensuring that the best generalising snapshot — not necessarily the final epoch — was used for evaluation.

3.5.5 Model Evaluation

The held-out test set was used for all final performance assessments. Metrics computed via Scikit-learn included **accuracy**, **precision**, **recall**, **F1-score**, and the **confusion matrix**. Both macro-averaged and weighted-averaged variants of the class-level metrics are reported, the former giving equal weight to each class regardless of frequency — a fairer measure when

minority class detection is a priority. ROC curves and AUC scores were computed for each class in a one-versus-rest fashion to assess discrimination capability independently of the classification threshold. This evaluation was replicated across all three datasets to assess cross-dataset generalisation.

3.6 Data Collection Methods

All data used in this study derive from publicly available benchmark datasets, each originally collected under controlled conditions designed to ensure the presence of both normal and malicious traffic in known proportions. The NSL-KDD data originates from the DARPA 1998 network simulation environment. CICIDS2017 was captured from a testbed that reproduced realistic user behaviour over five days of continuous collection, with attacks introduced at scheduled intervals (Sharafaldin et al., 2018). UNSW-NB15 was generated using the IXIA PerfectStorm tool in a controlled laboratory setting at the University of New South Wales.

No primary network data collection was undertaken. This decision reflects both practical constraints and principled methodological choices: using established datasets enables direct comparison with the existing literature, avoids the ethical complications of capturing live network traffic that may contain personal or sensitive data, and ensures that the results are reproducible by other researchers working from the same publicly available sources.

3.7 Data Analysis Techniques

The analytical work in this study spans three levels. **Descriptive analysis** characterises each dataset through summary statistics — means, standard deviations, ranges, and class frequency counts — and through visual tools including feature distribution histograms, box plots, and correlation heatmaps. This level of analysis informed preprocessing decisions and provided a basis for understanding the model's behaviour in light of the data it trained on.

Inferential analysis employed statistical tests to probe relationships within the data. The Kolmogorov-Smirnov test compared feature distributions between normal and attack traffic to identify features with strong discriminative potential. Chi-square tests assessed associations between categorical features and the target label, guiding feature selection decisions.

Predictive analysis forms the operational core of the study. The trained LSTM model's predictions are examined through confusion matrices that expose misclassification patterns at the class level — particularly revealing for understanding how the model handles the rare U2R and R2L categories. Permutation feature importance analysis identifies which features exert the greatest influence on predictions, offering interpretive insight that complements the aggregate performance metrics.

3.8 Safety and Ethical Considerations

This research was conducted in full accordance with established ethical principles for cybersecurity research. The exclusive use of publicly available benchmark datasets meant that no personally identifiable information was accessed or processed at any stage. The datasets were obtained from official repositories for academic research purposes and used strictly within those terms.

The study did not involve human participants, animal subjects, or any hazardous materials, and institutional ethics clearance was therefore not required. All software tools were used in compliance with their respective licences. Experiments were conducted in a secure, isolated offline environment, eliminating any risk of unintended network interference. In line with responsible disclosure principles recognised within the Nigerian academic research community (Akinwale & Adeyemi, 2020), all findings are intended solely to advance network security knowledge. The experimental code and results will be made available for academic verification subject to institutional policy. Data management practices throughout the study complied with the Nigerian Data Protection Regulation (NDPR), and GPU usage was managed with attention to minimising unnecessary energy consumption.